

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	Tharankeswaran M	Reg. No.:	192424081
Date:	30/7/26	Duration:	60 minutes

Code of Conduct

I Tharankeswaran M (192424081) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tharankeswaran M

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.

- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5

1 SYSTEM REQUIREMENT ANALYSIS

1.1 Problem Statement and Objectives

The **Smart Sports Stadium Operations Platform** must digitally transform stadium management by addressing six core objectives: (1) monitoring crowd movement in real time, (2) optimizing entry and exit flows, (3) coordinating security operations, (4) managing parking availability, (5) tracking concession performance, and (6) generating comprehensive event analytics.

1.2 Functional Requirements Influencing Architecture

- FR-1 **Real-Time Crowd Monitoring:** Ingest and process video streams and IoT occupancy data from >500 sensors/cameras with sub-second latency.
- FR-2 **Dynamic Flow Optimization:** Compute predictive queue models and push gate-staffing recommendations to digital signage controllers.
- FR-3 **Security Coordination:** Enable bidirectional mobile communication for incident reporting, dispatch, and escalation to external emergency services.
- FR-4 **Parking Management:** Track vehicle entry/exit across multiple zones and publish availability to mobile apps and LED displays.
- FR-5 **Concession Intelligence:** Integrate heterogeneous POS terminals, monitor inventory, and compute KPIs per stand.
- FR-6 **Event Analytics:** Aggregate multi-domain operational data into exportable post-event reports with comparative visualizations.
- FR-7 **Multi-Channel Delivery:** Serve web dashboards, native mobile apps, and digital signage from a unified backend.

1.3 Non-Functional Requirements Influencing Architecture

- NFR-1 **Performance:** End-to-end processing latency <2 seconds for 50,000 concurrent spectators.
- NFR-2 **Scalability:** Linear horizontal scaling from 10,000 to 100,000 spectator capacity without architectural change.
- NFR-3 **Availability:** 99.9% uptime during events with automatic failover and zero-downtime deployments.
- NFR-4 **Reliability:** MTBF >2,000 hours; MTTR <15 minutes through containerized auto-healing.
- NFR-5 **Security:** TLS 1.3 in transit, AES-256 at rest, OAuth 2.0 + MFA for administrative access.
- NFR-6 **Maintainability:** Independent deployability of subsystems; unit test coverage >80%.
- NFR-7 **Interoperability:** REST/GraphQL APIs, MQTT/CoAP for IoT, and WebSocket for real-time push.

1.4 Key Quality Attributes

Quality Attribute	Priority	Architectural Driver
Scalability	Critical	Must handle bursty sensor traffic during peak event periods.
Performance	Critical	Real-time safety alerts require sub-second decision pipelines.
Availability	Critical	System failure during an event risks spectator safety.
Reliability	High	Sensor and network failures must not cascade into system-wide outages.
Security	High	Sensitive operational data and video feeds require defense-in-depth.
Maintainability	High	Seasonal feature updates and vendor integrations demand loose coupling.

2 SELECTION OF SOFTWARE ARCHITECTURE

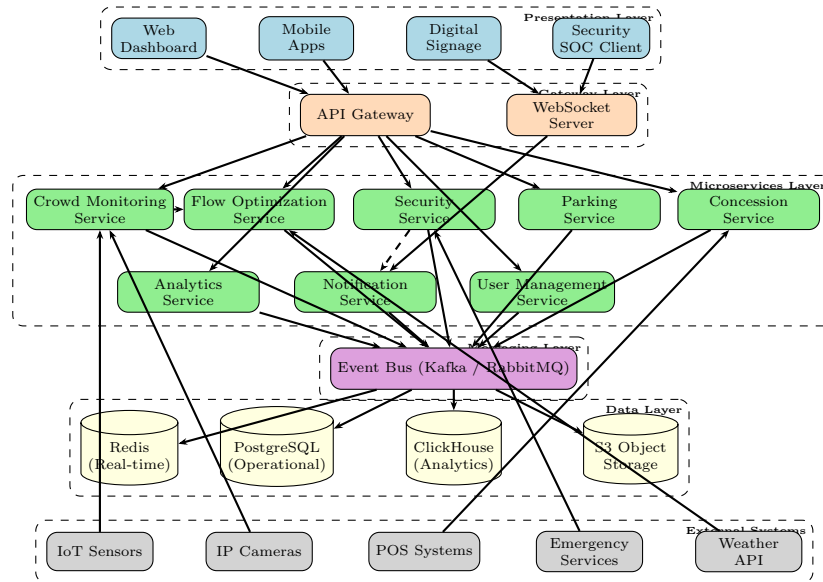
2.1 Proposed Architectural Style

The platform adopts a **Hybrid Microservices Architecture with Event-Driven and CQRS Patterns**. The design decomposes the monolithic stadium domain into independently deployable services, uses an event bus for asynchronous real-time communication, and separates read/write data paths to optimize for both transactional consistency and analytical query performance.

2.2 Justification of Selection

- **Domain Decomposition:** Crowd monitoring, parking, concessions, security, and analytics are distinct business capabilities with independent scaling needs. Microservices allow each domain to scale horizontally based on its own load profile.
- **Real-Time Responsiveness:** Event-driven messaging (Apache Kafka / RabbitMQ) decouples sensor ingestion from downstream processing, ensuring that a spike in camera data does not block parking or concession services.
- **Technology Heterogeneity:** Video analytics may require GPU-optimized Python services, while transactional POS integration may favor Java/Spring. Microservices permit polyglot persistence and polyglot programming.
- **Fault Isolation:** A failure in the concession analytics service must not compromise crowd-safety alerting. Service boundaries contain blast radius.
- **Independent Deployability:** Security patches or new analytics models can be deployed to individual services without a full-system release, satisfying the 6-month time-to-production constraint.
- **CQRS for Analytics:** Command paths handle incident logging and sensor writes; query paths serve dashboard aggregations and post-event reports, preventing analytical workloads from starving operational transactions.

3 ARCHITECTURE DIAGRAM AND COMPONENT DESIGN



3.1 Component Descriptions

Component	Responsibilities
API Gateway	Routes client requests, enforces rate limiting, validates JWT/OAuth tokens, and load-balances across service instances.
WebSocket Server	Maintains persistent connections for real-time push (alerts, heat-maps, parking updates) to dashboards and mobile apps.
Crowd Monitoring Service	Ingests video streams and sensor data; runs ML inference for density estimation; publishes crowd-event messages.
Flow Optimization Service	Computes queue-length predictions using historical and real-time data; generates gate-staffing recommendations.
Security Service	Manages incident lifecycle (report, dispatch, escalate); integrates with emergency services APIs.
Parking Service	Processes vehicle detection events; maintains zone occupancy counts; publishes availability updates.
Concession Service	Integrates POS transaction streams; monitors inventory thresholds; computes per-stand KPIs.
Analytics Service	Executes CQRS read queries; aggregates multi-domain data; generates post-event reports and visualizations.
Notification Service	Listens to critical events; dispatches SMS, push, and email alerts via third-party gateways.
User Management Service	Handles RBAC, authentication, MFA, session management, and audit logging.
Event Bus	Provides durable, ordered, fault-tolerant messaging between services; enables event sourcing for audit trails.
Redis	Hot cache for sub-second dashboard queries and session state; 24-hour TTL for real-time data.
PostgreSQL	ACID-compliant operational store for transactional data (incidents, users, configurations); 24-month retention.
ClickHouse	Columnar OLAP database optimized for analytical aggregations and time-series event queries.
S3 Object Storage	Archival for video footage, incident media, and exported reports; lifecycle policies for cost optimization.

4 COMPONENT INTERACTION AND WORKFLOW

4.1 Typical Workflow: Crowd Anomaly Detection and Response

1. **Ingestion:** IP cameras and IoT occupancy sensors stream data to the *Crowd Monitoring Service* via RTSP and MQTT.
2. **Processing:** The service runs GPU-accelerated computer vision models to compute density heat-maps every 5 seconds.
3. **Event Publication:** When density exceeds a configurable threshold, the service publishes a `CrowdCriticalEvent` to the Event Bus.
4. **Fan-Out:** The Event Bus fans the event to three subscribers:
 - *Notification Service* → pushes critical alerts to Security Officers' mobile apps and the SOC dashboard via WebSocket.
 - *Flow Optimization Service* → recalculates gate diversion recommendations and updates digital signage.
 - *Analytics Service* → writes the event to ClickHouse for trend analysis and writes a snapshot to PostgreSQL for incident correlation.
5. **Security Response:** A Security Officer receives the alert, opens the mobile incident-reporting module, and logs a response. The *Security Service* updates the incident state in PostgreSQL and, if severity warrants, calls the Emergency Services REST API.
6. **Cache Update:** Redis is updated with the latest crowd density so that the web dashboard reflects changes within 1 second.

4.2 Data Flow Patterns

- **Command Flow (Write):** Sensor data → Service → Event Bus → PostgreSQL (transactional) + ClickHouse (analytical) + Redis (cache).
- **Query Flow (Read):** Dashboard → API Gateway → Analytics Service → ClickHouse/Redis → Response.
- **Real-Time Push:** Event Bus → Notification Service → WebSocket Server → Client.

5 QUALITY ATTRIBUTE ANALYSIS

5.1 Scalability

Each microservice scales independently via container orchestration (Kubernetes). The *Crowd Monitoring Service* deploys additional GPU pods during high-attendance events, while the *Parking Service* remains at a baseline replica count. The Event Bus partitions topics by stadium zone, enabling horizontal scaling of consumers. ClickHouse shards analytical data by event date, ensuring query performance regardless of data volume growth.

5.2 Security

Defense-in-depth is implemented at every layer:

- **Edge:** API Gateway terminates TLS 1.3, performs WAF filtering, and enforces OAuth 2.0 / OIDC authentication.
- **Service:** Inter-service communication uses mTLS (mutual TLS) with SPIFFE/SPIRE identity.
- **Data:** AES-256 encryption at rest for all databases and object storage; field-level encryption for PII in PostgreSQL.
- **Access Control:** RBAC with five operational roles; MFA enforced for administrators and SOC operators.

5.3 Performance

Redis caching eliminates repeated database hits for dashboard aggregates. ClickHouse’s columnar storage delivers sub-second analytical queries over billions of rows. GPU inference in the *Crowd Monitoring Service* processes 30 fps video streams with <100 ms inference latency. Asynchronous event publishing prevents I/O blocking on the critical path.

5.4 Reliability

Circuit breakers (e.g., Resilience4j) prevent cascading failures when external systems (POS, weather API) are unreachable. The Event Bus persists messages with replication factor ≥ 3 , ensuring zero message loss during broker failures. Health-check probes and auto-restart policies in Kubernetes guarantee MTTR <15 minutes.

5.5 Maintainability

Each microservice owns its codebase, CI/CD pipeline, and database schema. Loose coupling via the Event Bus means that replacing the video analytics engine (e.g., switching from TensorFlow to PyTorch) affects only the *Crowd Monitoring Service*. Comprehensive OpenAPI specifications and centralized logging (ELK stack) simplify debugging and onboarding.

5.6 Availability

Multi-zone Kubernetes clusters with pod anti-affinity rules distribute services across availability zones. Database clusters use synchronous replication for PostgreSQL and asynchronous replication for ClickHouse. Blue-green deployments via the API Gateway allow zero-downtime releases. The 99.9% SLA is further protected by automated failover of the WebSocket Server to standby instances.

6 ARCHITECTURAL JUSTIFICATION AND TRADE-OFFS

6.1 Rationale for Key Decisions

- **Microservices over Monolith:** A monolith would simplify initial deployment but would fail the scalability and availability requirements. Stadium events create extreme load spikes that monolithic thread pools cannot isolate. Microservices provide the necessary fault boundaries.
- **Event-Driven over Synchronous REST:** Synchronous chaining between crowd detection, notification, and flow optimization would introduce unacceptable latency and tight coupling. The Event Bus decouples producers from consumers, enabling independent evolution.
- **CQRS over Single Data Store:** A single relational database would bottleneck under the mixed workload of high-frequency sensor writes and complex analytical reads. CQRS with PostgreSQL (OLTP) and ClickHouse (OLAP) optimizes each access pattern.

- **Kafka over Simple Message Queue:** Apache Kafka (or equivalent) is chosen for its durability, replay capability, and support for event sourcing—critical for post-incident forensic reconstruction.

6.2 Trade-offs Considered

Decision	Advantage Gained	Cost / Complexity
Microservices	Independent scaling and deployment	Operational overhead: distributed tracing, service mesh, DevOps maturity.
Event Bus	Loose coupling and resilience	Eventual consistency; requires idempotency and outbox patterns.
Polyglot Persistence	Optimal data model per use case	Increased operational expertise needed (Redis, PostgreSQL, ClickHouse, S3).
GPU Inference	High-accuracy video analytics	Higher infrastructure cost; specialized ML-ops pipeline.

6.3 Future Enhancements and Integration

- **Digital Twin Integration:** The event-sourced architecture naturally supports replaying historical event streams to train stadium digital twins for predictive scenario modeling.
- **AI-Driven Dynamic Pricing:** The *Concession Service* can be extended with reinforcement-learning modules without modifying other domains.
- **Multi-Stadium Federation:** The API Gateway and user service already support multi-tenant routing, enabling a SaaS model for stadium chains.
- **5G Edge Computing:** Camera inference can be migrated to edge nodes; the Event Bus architecture accommodates edge-to-cloud message bridging with minimal change.

6.4 Long-Term Maintainability

The architecture adheres to the *Single Responsibility Principle* and *Domain-Driven Design* bounded contexts. Each service exposes versioned REST/GraphQL APIs, ensuring backward compatibility during iterative releases. Infrastructure-as-Code (Terraform) and GitOps (ArgoCD) guarantee reproducible environments, reducing configuration drift and technical debt over the platform’s lifecycle.